

PLLM: Composable Multi-Party Inference and Autonomous Research for Private LLMs

Blair Hudson
deployscience labs
blair@deployscience.com

16 September 2026

Abstract

PLLM is a high-performance private LLM multi-party inference runtime and autonomous research harness. Its implemented prepared runtime separates Client, Preparation, and Inference. Preparation derives one-use masks and uploads $Wr - s$ corrections before generation; online, Client and Inference exchange only tickets, masked activations $x - r$, and masked results $Wx - s$. A trusted local gateway retains plaintext, model state, and decoding inside the client boundary. The research harness lowers model families into a shared semantic representation, groups independently reimplemented methods by capability, composes compatible components into immutable plans, and binds those plans to assurance and benchmark evidence. Constrained plan-space search is planned, not yet implemented. Adapter coverage likewise does not imply complete execution: the current compiler cannot execute a complete lowered model, and garbling is limited to experimental bounded Q7 SiLU and scalar gated-multiply components. Security requires protocol-following, non-colluding Preparation and Inference roles. Retained performance evidence covers only nine historical warm Qwen2.5-0.5B CPU-loopback runs and supports no broader deployment or performance claim.

Introduction

Ordinary hosted inference gives one operator the user’s plaintext prompt, token identities, intermediate state, and generated output. Transport encryption protects these values in transit but not from the endpoint performing inference. Private neural-inference systems instead distribute computation using techniques such as homomorphic encryption, secret sharing, and secure two-party computation (Huang et al. 2022; Chen et al. 2022; Hao et al. 2022).

PLLM combines two connected systems. Its high-performance private LLM multi-party inference runtime retains language and nonlinear state in a client-controlled environment while remote services perform masked integer matrix work. Its autonomous research harness represents model semantics, admits independently reimplemented components through typed capability contracts, composes compatible plans, and binds evaluation evidence to the exact plan tested. A complete semantic plan is not evidence of complete executable private inference.

This paper claims an implemented prepared three-role runtime, a trusted local gateway, semantic adapters for the listed Qwen, Phi, and Gemma configurations, typed component and plan foundations, and a bounded experimental Q7 SiLU garbling component. It does **not** claim complete model execution through the compiler or implemented autonomous plan search. Evaluation is restricted to retained historical measurements from the prepared runtime.

Prepared inference protocol

Roles and application boundary

The **Client** is trusted. It owns plaintext input and output, tokenization, one-time root seeds and masks, private activation scales, attention and other nonlinear state, sampling, and decoding. For public runtime bundles it also evaluates token lookup and the output head locally. The optional **p11m gateway** runs on loopback inside this boundary. Applications submit ordinary Responses API or Chat Completions API requests to that gateway, which invokes the private PLLM client path. Preparation and Inference are protocol services, not application-facing OpenAI-compatible endpoints.

Preparation is trusted with the public transformer body and offline seeds. It must follow the protocol, erase expanded masks, and remain independent of Inference. **Inference** holds the same public body and performs on-line matrix work, but is not trusted with plaintext client values. The intended privacy boundary assumes both remote roles are honest-but-curious and do not collude.

Offline preparation and online use

For public quantized matrix W and private integer activation x , the Client sends Preparation a fresh root seed and row count for each remote stage. Domain-separated expansion binds derived values to the inventory, model body, stage, weight, shape, quantization, ring, modulus, and wire width. Client and Preparation thereby obtain

fresh input mask r , output mask s , and ticket for each row. Preparation computes

$$c = Wr - s$$

and pushes c to Inference before the inventory becomes ready. Inference acknowledges accepted stage batches and seals the inventory only after every committed stage has loaded. Inference receives neither root seeds nor masks; the Client does not receive c .

Online, the Client sends Inference a one-use ticket and masked activation $x - r$. Inference atomically consumes the matching correction and returns

$$W(x - r) + c = W(x - r) + (Wr - s) = Wx - s.$$

The Client adds s and center-decodes Wx . Preparation receives no online message. Fresh pseudorandom r masks x from Inference, while fresh s masks the offline correction. Reusing a row would reveal relations between activations, so reuse is forbidden.

Inventory lifecycle

An inventory commits to model identity, immutable body fingerprint, complete remote stage set, quantization parameters, arithmetic ring, wire width, and an attempt budget. A response atomically reserves disjoint rows. Completion, cancellation, failure, replay, or early termination burns every unused row in the reservation. Restart, replacement, and idle expiry discard in-memory inventory.

PreFill sends a ticket vector and packed masked matrix for each stage rather than one envelope per prompt row. Decode uses one ticket per stage over a persistent authenticated connection. Stages select the smallest exact 2^{16} , 2^{24} , or 2^{32} ring justified by their signed output bound. These are runtime engineering properties; they do not strengthen the threat model.

Threat model and limits

Inference does not receive literal prompt text, token IDs, decoded output, root seeds, masks, private activation scales, local attention state, or sampling choices. Preparation receives seeds and can derive masks, but does not receive online masked activations. Under fresh pseudorandom masks, authenticated channels, protocol-conforming execution, Preparation erasure, and non-collusion, neither remote role alone has both views needed to reconstruct client activations.

The split fails if Preparation and Inference collude: Preparation’s r and Inference’s $x - r$ reconstruct x . Two services controlled by one operator do not establish meaningful non-collusion. Self-hosting Preparation keeps its trust in the client environment, but co-locating all roles is only a functional topology, not evidence of independent operators.

Both services observe the public model, stage identities, tensor shapes, quantization, timing, traffic volume, and approximate sequence length. Authentication and replay controls do not prove correct execution, administrative independence, or secure deletion. The protocol does not defend against arbitrary malicious participants, and physical zeroization of Preparation’s Python memory has not been established. Audit counters showing no literal plaintext bytes are implementation observations, not a proof against side channels. The local gateway and its host remain trusted.

Implementation status

Prepared runtime

PLLM is distributed as one Python package with Rust native crates. In the prepared runtime, Python owns model orchestration, checkpoint import, protocol lifecycle, transport, scheduling, and gateway behavior. Rust owns validated immutable integer matrices, bounded arithmetic, codecs, randomness, scalar reference paths, runtime SIMD selection, and a persistent worker pool. Matrices are quantized and copied into Rust once for repeated execution. The current public-weight online path uses prepared additive masks, not homomorphic encryption.

This runtime is client-heavy: attention, normalization, nonlinear operations, token boundaries, state, and sampling remain local. It supports prepared generation for selected text checkpoint layouts and has end-to-end historical evidence for Qwen2.5-0.5B. That fact is independent of the newer semantic compiler.

Semantic IR and compiler

Model adapters lower architecture-specific configuration into a model-family-neutral decoder IR. Operators, layer identity, numeric meaning, workload bounds, and persistent-state kinds are explicit. Compiler and research-method passes select semantic operators and declared capabilities instead of parsing adapter-specific node names or parameter paths. Unsupported coverage fails closed rather than silently moving a protected region to plaintext execution.

Implemented adapters cover dense Qwen2, dense Qwen3, the pinned Qwen3.5-4B text decoder, Phi-4-mini-instruct, and the official Gemma 4 E2B and E4B text configurations. They represent fused projections, rotary variants, shared KV state, and, where applicable, recurrent and convolution state in the shared IR. Adapter support means configuration validation and semantic lowering only. It does not establish checkpoint import, compiler operator coverage, executable distributed placement, numerical parity, generation quality, or deployment support.

The current named compiler profile reports incomplete coverage and cannot execute any complete listed model plan. In particular, complete protected attention, normalization, rescaling and truncation, persistent state, token selection, sampling, and feedback are not jointly available

as an executable plan. Existing prepared-runtime model execution must therefore not be described as execution of the new compiler’s `ModelPlan`.

Component library and autonomous search

Research methods enter PLLM as implementations of stable capability families, not as isolated paper-shaped runtime paths. Cache policy, numeric approximation, non-linear protocol, matrix protocol, preparation, kernel, and placement components declare their semantic input and output, numeric domain, state, placement, and security requirements. Like-for-like alternatives share a module; new families can be introduced when research adds a genuinely different contract. Immutable plans record selected implementations and transformation lineage.

The autonomous harness is intended to turn pinned sources and hypotheses into validated components, compose only compatible candidates, execute matched benchmarks, and retain failures as evidence. Current plan, benchmark, assurance, and artifact interfaces establish that foundation. Grid search over small discrete spaces and seeded random search over larger spaces are planned; neither exists in the current runtime, and no candidate is promoted without separate correctness, security, quality, and performance gates.

Experimental Q7 gated-MLP garbling

Garbling support is narrower still. The compiler can bind and evaluate a one-use arithmetic-garbling payload for at most 128 elements of signed Q7 SiLU input over $[-1, 1]$. It computes the fixed quadratic approximation $q(x) = x/2 + x^2/4$ with deterministic ties-to-even rounding; the implementation’s exhaustive encoded-domain test bounds absolute error against SiLU by 0.02285. Payloads are digest-bound, strictly decoded, and burned through a bounded process-local ledger.

A separate scalar reference region jointly garbles SiLU and two-input Q7 multiplication. The SiLU output remains an encoded label and enters the binary gate directly; it is never decoded and re-encoded between operators. Both inputs are provenance-bound to exact dense-Qwen linear-to-Q7 edges. The binary table is about 3.18 MB per scalar, so tensor execution remains deliberately unavailable.

This component is an experimental primitive, disabled from complete deployment profiles. It is not full-model garbling, does not cover the other missing operators at tensor scale, and has no cryptographic review or production-security claim. It also does not replace the additive-masking protocol used by the prepared runtime.

Historical evaluation

Method

The retained evidence predates the compiler and adapter work above. It records PLLM revision

277d19f, Python 3.13.15, and Qwen2.5-0.5B-Instruct revision 7ae557604adf67be50417f59c2c2f167def9a775 (Qwen Team 2024) on one Apple M5 CPU host with 32 GiB memory. Client, Preparation, and Inference ran as separate loopback processes. One excluded warmup preceded nine warm measurements: three runs at each exact 30-, 63-, and 255-token input length. Output was capped at 16 tokens. Only records with an authoritative `response.completed` event and settled telemetry were accepted.

Latency. Table 1 reports cohort medians; parentheses contain all three TTFT observations. Online time starts after inventory readiness, whereas full time includes preparation and transition overhead. The 30-token cohort ended after nine output tokens; the other cohorts produced 16, so throughput is not compared.

Traffic and preparation. Client I/O grows with activation rows and stage width. Correction bytes moved from Preparation to Inference before online timing. Stage rows count aggregate matrix work across remote body stages.

All nine records report zero Preparation requests and protocol operations during the online interval, and zero literal plaintext prompt or token-ID bytes in the instrumented counters. These observations validate the measured implementation path, not a general privacy theorem. Preparation remains real computation and traffic despite occurring outside online timing.

The study has three small cohorts on one CPU loopback host and no concurrent load, physical network, accelerator, model-quality, malicious-provider, or secure-erasure evaluation. It must not be used to infer current compiler performance, other-model performance, operating cost, or deployment security.

Conclusion

PLLM is a high-performance private LLM multi-party inference runtime and autonomous research harness. Its current runtime implements prepared public-weight inference with an explicit three-role boundary: Client sends seeds to trusted Preparation offline; Preparation sends $Wr - s$ corrections to Inference; Client and Inference exchange one-use tickets and masked activations online. A trusted loopback gateway presents familiar Responses and Chat Completions interfaces without moving plaintext out of the client boundary.

The harness also lowers several model families into a shared semantic IR and organizes research implementations as composable capability families. Complete plan-compiled model execution and autonomous plan search remain unavailable. Garbled nonlinear execution is limited to bounded experimental Q7 SiLU and one scalar label-preserving gated-multiply composition. These runtime, compiler, component, and evidence claims remain separate. Retained Qwen2.5 measurements show one historical CPU-loopback implementation working within that boundary; they establish nothing beyond that recorded system and workload.

Table 1: Retained historical CPU-loopback latency; three runs per cohort.

Input	Output	TTFT (range), s	Online, s	Full, s
30	9	0.978 (0.935–0.982)	2.618	4.674
63	16	1.374 (1.353–1.379)	3.194	5.347
255	16	5.158 (4.961–5.289)	7.442	13.059

Table 2: Retained historical lifecycle traffic and offline preparation.

Input	Client I/O	Correction push	Stage rows
30	63.33 MB	59.80 MB	6,144
63	126.29 MB	72.88 MB	7,488
255	432.71 MB	252.18 MB	25,920

References

- Chen, Tianyu, Hangbo Bao, Shaohan Huang, et al. 2022. “THE-X: Privacy-Preserving Transformer Inference with Homomorphic Encryption.” *Findings of the Association for Computational Linguistics: ACL 2022*, 3510–20. <https://doi.org/10.18653/v1/2022.findings-acl.277>.
- Hao, Meng, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. “Iron: Private Inference on Transformers.” *Advances in Neural Information Processing Systems 35*, 15718–31. <https://doi.org/10.52202/068431-1143>.
- Huang, Zhicong, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. “Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference.” *31st USENIX Security Symposium*, 809–26. <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong>.
- Qwen Team. 2024. *Qwen2.5 Technical Report*. arXiv:2412.15115. <https://doi.org/10.48550/arXiv.2412.15115>.